
UNIT 3 PYTHON'S CONTROL FLOW STRUCTURES

Structure

- 3.1 Introduction
- 3.2 Objectives
- 3.3 Control Flow Statements
 - 3.3.1 Sequential Control Flow Statements
 - 3.3.2 Decision Control Flow Statements
 - 3.3.2.1 The if Decision Control Flow Statement
 - 3.3.2.2 The if...else Decision Control Flow Statement
 - 3.3.2.3 The if...elif...else Decision Control Statement
 - 3.3.2.4 Nested if Statement
 - 3.3.3 Loop Control Flow Statements
 - 3.3.3.1 The while loop
 - 3.3.3.2 The for loop
 - 3.3.3.3 The continue and break Statements
- 3.4 Summary
- 3.5 Solution to Check your Progress
- 3.6 Reference/Further Readings

3.1 INTRODUCTION

In the previous unit, we have mainly seen programming examples in which programs execute line by line, top to bottom—following a strict, linear path. However, real-world problems are rarely that straightforward. Programs often need to make decisions based on changing data, repeat tasks efficiently, or skip/jump unnecessary steps. In this unit, you will learn about Control Flow Statements. These are the core mechanisms that direct the order in which code executes. You will explore how to transition from basic sequential execution to decision-making structures (if, if...else, if...elif...else, and nested conditions) that dynamically alter execution paths. In this unit, you will also learn to handle repetitive logic using loops (for and while) alongside control keywords like break and continue. Understanding these fundamental control structures will help you build flexible, efficient, and logical Python programs that can handle complex decision-making processes.

3.2 OBJECTIVES

After going through this unit, you will be able to:

- Understand usage of sequential control flow statements,
- Use decision structures in programming,
- Use nested decision making mechanisms in programming,
- Use looping constructs in programming, and

- Use continue and break statements.

3.3 CONTROL FLOW STATEMENTS

Python supports a set of control flow statements that you can integrate into your program. The statements inside your Python program are generally executed sequentially from top to bottom in the order that they appear. Apart from sequential control flow statements, you can use decision-making and looping control flow statements to break up the flow of execution, thus enabling your program to conditionally execute a particular block of code. Control flow describes the direction the program takes.

The control flow statements in the Python Programming Language are:

3.3.1 Sequential Control Flow Statements :

This refers to the line-by-line execution, in which the statements are executed sequentially, in the same order in which they appear in the program.

Example: Taking three values as input and displaying their sum

```
# Prompt the user to enter three numbers
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))
num3 = float(input("Enter the third number: "))
# Calculate the sum
total_sum = num1 + num2 + num3
# Display the result
print(f"The sum of {num1}, {num2}, and {num3} is: {total_sum}")
```

The above program is an example of sequential control flow. Also, one important point to note in the above program is that in line:

```
print(f"The sum of {num1}, {num2}, and {num3} is: {total_sum}")
```

The **f** stands for Formatted String Literal, commonly known as an **f-string**.

When you put an **f** right before the opening quote of a string, it tells Python: "Look inside this string for curly braces {} and replace them with the actual values of the variables inside them."

The screenshot shows the PyCharm IDE with a project named 'PythonProject2 [Test1]'. The file explorer on the left shows files: 'main.py', 'SequentialStatement.py', 'UseFrozenSet.py', and 'FrozenSet.py'. The main editor displays the code in 'SequentialStatement.py':

```

1 # Prompt the user to enter three numbers
2 num1 = float(input("Enter the first number: "))
3 num2 = float(input("Enter the second number: "))
4 num3 = float(input("Enter the third number: "))
5 # Calculate the sum
6 total_sum = num1 + num2 + num3
7 # Display the result
8 print(f"The sum of {num1}, {num2}, and {num3} is: {total_sum}")

```

The Run console at the bottom shows the execution output:

```

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\SequentialStatement.py
Enter the first number: 1
Enter the second number: 2
Enter the third number: 4
The sum of 1.0, 2.0, and 4.0 is: 7.0
Process finished with exit code 0

```

3.3.2 Decision Control Flow Statements :

Depending on whether a condition is True or False, the decision structure may skip the execution of an entire block of statements or even execute one block of statements instead of another. Python has *if*, *if...else* and *if...elif...else* statements for conditional control flow of execution.

3.3.2.1 The if Decision Control Flow Statement :

In order to write useful programs, we almost need the ability to check conditions and change the behaviour of the program accordingly. The Decision Control Flow Statements give us this ability. The simplest form is the *if* statement.

The syntax for *if* statement is:

```

if Boolean_Expression :
    statement (s)

```

The Boolean_Expression after the *if* statement is called the condition. We end the *if* statement with a colon character (:) and the line(s) after the *if* statement are indented. The *if* statement decides whether to run statement (s) or not depending upon the value of the Boolean expression. If the Boolean expression evaluates to True, then the indented statements in the *if* block will be executed; if the result is False, then none of the statements is executed. e.g.,

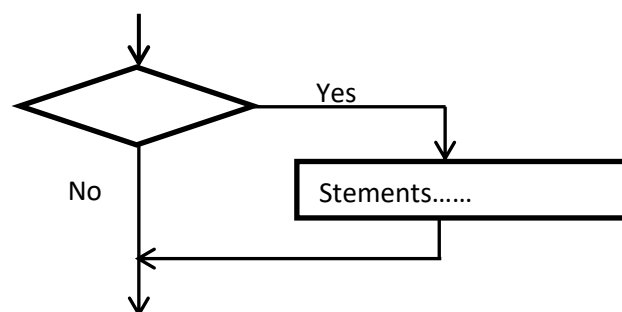


Fig. 3.1: If Logic

```
if x > 0 :  
  
    print ( 'x is positive' )
```

Here, depending on the value of x, the print statement will be executed, i.e. only if $x > 0$.

There is no limit on the number of statements that can appear in the body, but there must be at least one. In Python, the if block statements are determined through indentation, and the first unindented statement marks the end. You don't need to use the == operator explicitly to check if the variable's value evaluates to True, as the variable name can itself be used as a condition.

Program: Program reads your age and prints a message whether you are eligible to vote or not.

```
age = int(input("Enter your age : "))  
if age >= 18 :  
    print ("Congratulations!!! you can vote")  
    print ("!!! Thanks !!!")  
    print ("Voting is your birth right. Use judicially")  
  
age = int(input("Enter your age : "))  
if age >= 18 :  
    print ("Congratulations!!! you can vote")  
    print ("!!! Thanks !!!")  
    print ("Voting is your birth right. Use judicially")  
  
Enter your age : 22  
Congratulations!!! you can vote  
!!! Thanks !!!  
Voting is your birth right. Use judicially
```

Above you can see, the condition or Boolean expression is true, indented execution program would display:

```
Enter your age : 17  
Voting is your birth right. Use judicially
```

Above, the condition is False, so the indented block would not be executed and only the unindented statement after the if block would be executed.

3.3.2.2 The if...else Decision Control Flow Statement

An if statement can also be followed by an else statement (which is optional). An else statement does not have any condition. Statements in the if block are executed if the Boolean_Expression is True. Use the optional else block to execute statements if the Boolean_Expression is False. The if...else statements allow for a two-way decision.

The syntax for *if...else* statement is:

```
if Boolean_Expression :
    statement_blk_1
else
    statement_blk_2
```

If the Boolean_Expression evaluates to True, then statement_blk_1 (single or multiple statements) is executed, otherwise it is evaluated to False then statement_blk_2 (single or multiple statements) is executed. Indentation is used to separate the blocks. After the execution of either statement_blk_1 or statement_blk_2, the control is transferred to the next statement after the if statement. Also, if and else keywords should be aligned at the same column position.

Program 10.6 : Program to find if a given number is odd or even

```
num = int(input("Enter a number : "))
if num % 2 == 0 :
    print (num," is even ")
else:
    print (num," is odd ")
```

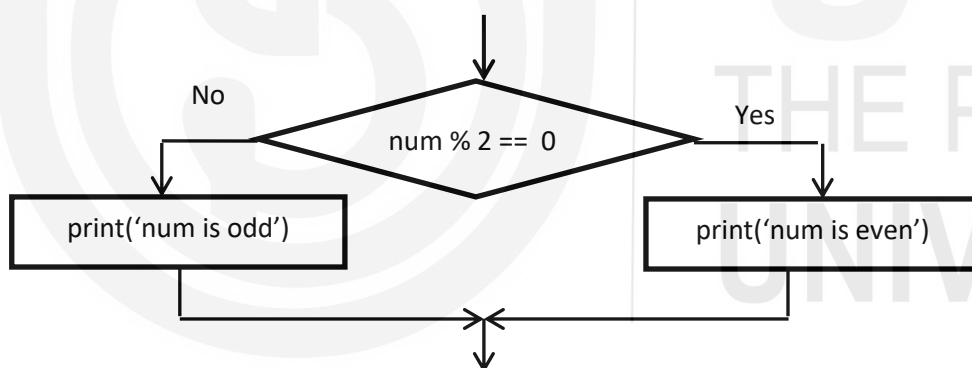


Fig. 3.2: if-else Logic

A number is read and stored in the variable named num. The num is checked using modulus operator to determine whether the num is perfectly divisible by 2 or not. The num entered is 15, which makes the evaluated expression False, so the else statement is executed, and num is odd.

```
num = int(input("Enter a number : "))
if num % 2 == 0:
    print (num," is even number")
else:
    print (num," is odd number")
```

```
Enter a number : 15
15 is odd number
```

3.3.2.3 The if...elif...else Decision Control Statement

The *if...elif...else* is also called a multi-way decision control statement. When you need to choose from several possible alternatives, an elif statement is used along with an if statement. The keyword 'elif' is short for 'else if' and is useful to avoid excessive indentation. The else statement must always come last, and will again act as the default action.

The syntax for if...elif...else statement is,

```
if Boolean_Expression_1 :  
    statement_blk_1  
elif Boolean_Expression_2 :  
    statement_blk_2  
elif Boolean_Expression_3 :  
    statement_blk_3  
:  
:  
:  
else :  
    statement_blk_last
```

This if...elif...else decision control statement is executed as follows:

- In the case of multiple Boolean expression, only the first logical Boolean expression which evaluates to True will be executed.
- If Boolean_Expression_1 is True, then statement_blk_1 is executed.
- If Boolean_Expression_1 is False and Boolean_Expression_2 is True, then statement_blk_2 is executed.
- If Boolean_Expression_1 and Boolean_Expression_2 are False and Boolean_Expression_3 is True, then statement_blk_3 is executed and so on...
- If none of the Boolean_Expression is True, then statement_blk_last is executed.

Program: Write a program to prompt for a score between 0.0 and 1.0. If the score is out of range, print an error. If the score is between 0.0 and 1.0, print a grade using the following table.

Score	Grade
>= 0.9	A
>= 0.8	B
>= 0.7	C
>= 0.6	D
< 0.6	F

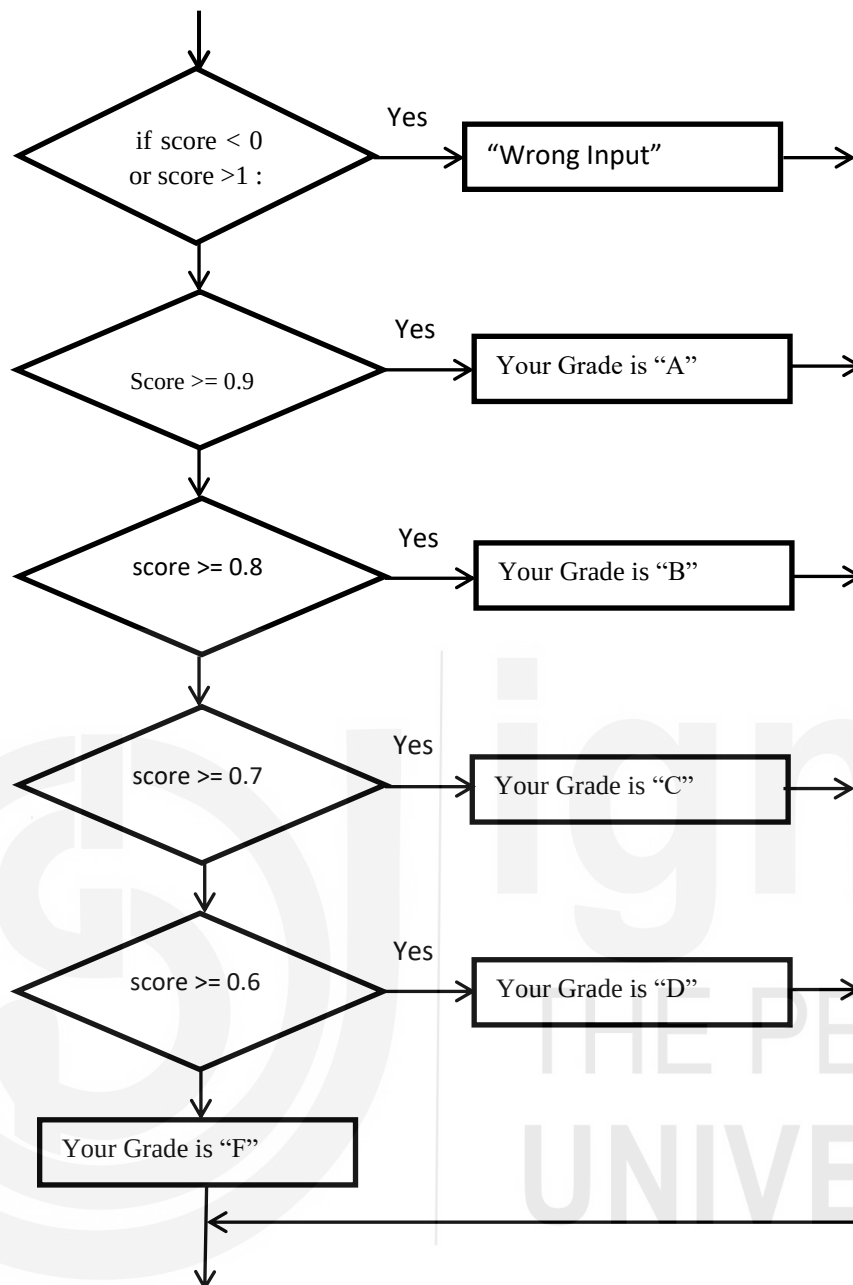
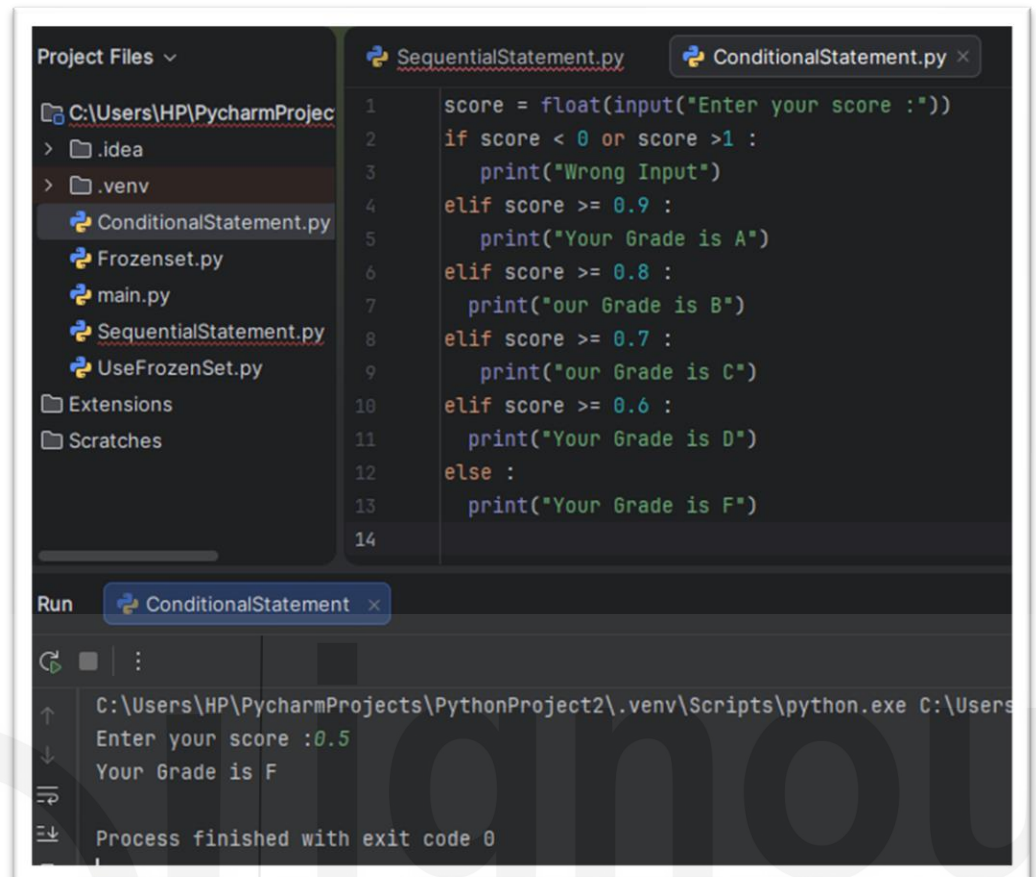


Fig. 3.3 : Flowchart of the Program

```
score = float(input("Enter your score :"))
if score < 0 or score > 1 :
    print("Wrong Input")
elif score >= 0.9 :
    print("Your Grade is A")
elif score >= 0.8 :
    print("our Grade is B")
elif score >= 0.7 :
    print("our Grade is C")
elif score >= 0.6 :
    print("Your Grade is D")
else :
    print("Your Grade is F")
```



```
Project Files
C:\Users\HP\PycharmProjec
> .idea
> .venv
ConditionalStatement.py
FrozenSet.py
main.py
SequentialStatement.py
UseFrozenSet.py
Extensions
Scratches

SequentialStatement.py
1 score = float(input("Enter your score :"))
2 if score < 0 or score >1 :
3     print("Wrong Input")
4 elif score >= 0.9 :
5     print("Your Grade is A")
6 elif score >= 0.8 :
7     print("our Grade is B")
8 elif score >= 0.7 :
9     print("our Grade is C")
10 elif score >= 0.6 :
11     print("Your Grade is D")
12 else :
13     print("Your Grade is F")
14

Run ConditionalStatement x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users
Enter your score :0.5
Your Grade is F
Process finished with exit code 0
```

3.3.2.4 Nested if Statement

In some situations, you have to place an if statement inside another statement. An if statement that contains another if statement either in its if block or else block is called a **Nested if statement**.

he syntax of the nested if statement is:

```
if Boolean_Expression_1 :
    if Boolean_Expression_2 :
        statement_blk_1
    else :
        statement_blk_2
else :
    statement_blk_3
```

If the Boolean_Expression_1 is evaluated to True, then the control shifts to Boolean_Expression_2, and if this expression is evaluated to True, then statement_blk_1 is executed. If Boolean_Expression_2 is evaluated to False, then statement_blk_2 is executed. If the Boolean_Expression_1 is evaluated to False, then the statement_blk_3 is executed.

A three-branch example could be written as :

```
if x == y :
    print('x and y are equal')
```

```

else :
    if x < y :
        print('x is less than y')
    else :
        print('x is greater than y')

```

The outer conditional contains two branches. The first branch contains a simple statement. The second branch contains another if statement has two branches of its own. Those two branches are both simple statements, although they could have been conditional statements as well.

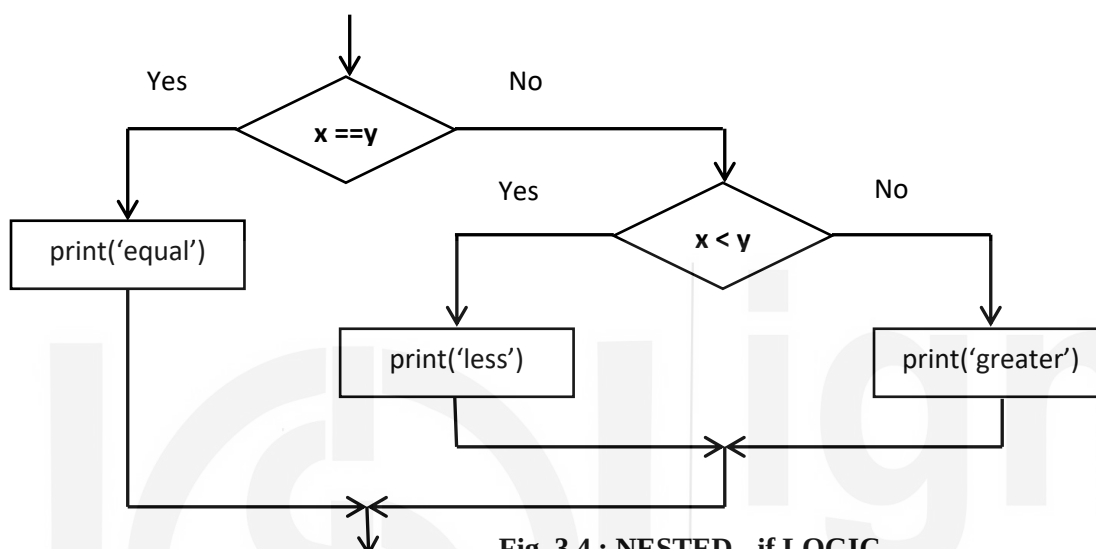


Fig. 3.4 : NESTED - if LOGIC

Program to check if a given year is a leap year

Note: All years which are perfectly divisible by 4 are leap years except for century years (years ending with 00), which are leap years only if they are perfectly divisible by 400. For example, years like 2012, 2004, 1968 are leap years, but 1971, 2006 are not leap years. Similarly, 1200, 1600, 2000, 2400 are leap years, but 1700, 1800, 1900 are not.

Screen shot of execution of Program :

```

year = int(input('Enter a year : '))
if year % 4 == 0:
    if year % 100 == 0 :
        if year % 400 == 0 :
            print(f'{year} is a Leap Year')
        else :
            print(f'{year} is not a Leap Year')
    else :
        print(f'{year} is a Leap Year')
else :
    print(f'{year} is not a Leap Year')

```

```

Enter a year : 2014
2014 is not a Leap Year

```

Check your progress -1

- 1) Write a program to take a number as input and find whether the number is odd or even and positive or negative.

- 2) Write a program to calculate bonus for employees based on basic pay, DA, HRA and grade of the employee. Make necessary assumptions.

3.3.3 Loop Control Flow Statements

These control structures allow the execution of a block of statements multiple times until a loop termination condition is met. Loop Control Flow statements are also called Repetition statements or Iteration Statements.

3.3.3.1 The *while* loop

Computers are often used to automate repetitive tasks. Repeating identical or similar tasks without making errors is something that computers do well, and people do poorly. Because iteration is so common, Python provides for loop and while loop constructs for writing iterative programming blocks.

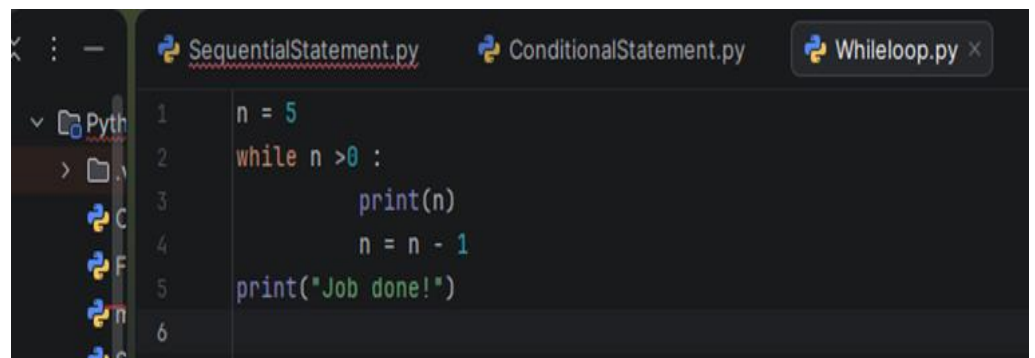
The syntax for while loop is :

```
while Boolean_Expression :  
    statement(s)
```

Example: Program that counts down from five and then says "Job done!"

```
n = 5  
while n > 0 :  
    print(n)  
    n = n - 1  
print('Job done!')
```

Screenshot of execution of Program

A screenshot of a Python IDE with a dark theme. The top bar shows three open files: 'SequentialStatement.py', 'ConditionalStatement.py', and 'Whileloop.py'. The 'Whileloop.py' file is active, showing a Python script with five lines of code: 1. n = 5, 2. while n > 0 :, 3. print(n), 4. n = n - 1, 5. print("Job done!"). The left sidebar shows a file explorer with a 'Pyth' folder expanded, containing subfolders for 'C', 'F', 'n', and 'S'.



```

Run Whileloop x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users
5
4
3
2
1
Job done!
Process finished with exit code 0

```

You can almost read the while statement as if it were English. It means, “While n is greater than 0, display the value of n and then reduce the value of n by 1. When you get to 0, exit the while statement and display the “Job done”.

More formally, the flow of execution for a while statement is :

1. Evaluate the condition or the Boolean_Expression, yielding True or False.
2. If the Boolean_Expression is false, exit the while statement and then continue execution at the next statement.
3. If the Boolean_Expression is true, execute the body and then go back.

This flow is called a loop because the third step loops back to the top. We call each time we execute the loop body an iteration. For the above loop, we would say, “It had five iterations”, which means that the body of the loop was executed five times.

The loop body should change the value of one or more variables so that the condition, or Boolean_Expression, eventually becomes false and the loop terminates. We call the variable that changes each time the loop executes and controls when the loop finishes the iteration variable. If there is no iteration variable, the loop will **repeat forever**, resulting in an **infinite loop**.

Example: Program to display the Fibonacci sequence up to the nth term, where n is taken from the user as input.

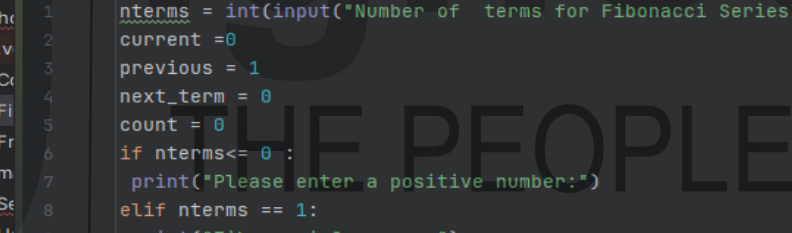
Note: In a Fibonacci sequence, the next number is obtained by adding the previous two numbers. The first two numbers of the Fibonacci sequence are 0 and 1. The next number is obtained by adding 1 and 1, which is 1. Again, the next number is obtained by adding 1 and 1, which is 2 and so on. Get a number from the user up to which you want to generate the Fibonacci sequence. Assign values to variables current, previous, next_term and count. The variable count keeps track of the number of times the while block is executed. The user is required to enter a positive number to generate a single number in the sequence, then print zero. The next_term is obtained by adding the previous and current variables, and the statements in the while block are repeated until the while block conditional expression becomes False.

Program:

```

nterms = int(input('How many terms?'))
current = 0

```



```
SequentialStatement.py ConditionalStatement.py Whileloop.py
Python
> .v
> Cc
> Fi
> Fr
> m
> Se
> U:
> W
> Exter
> Scrat

1 nterms = int(input("Number of terms for Fibonacci Series?"))
2 current = 0
3 previous = 1
4 next_term = 0
5 count = 0
6 if nterms <= 0 :
7     print("Please enter a positive number:")
8 elif nterms == 1:
9     print("Fibonacci Sequence:")
10    print("0")
11 else:
12    print("Fibonacci Sequence:")
13    while count < nterms :
14        print(next_term)
15        current = next_term
16        next_term = previous + current
17        previous = current
18        count += 1
19
```

12

3.3.3.2 The for loop

We call the while statement an indefinite loop because it simply loops until some condition becomes false, whereas the for loop is looping through a known set of items so it runs through as many iterations as there are items in the set. Sometimes we want to loop through a set of things such as a list of words, the lines in a file, or a list of numbers. In other words, we use a *for loop* when we want to repeat a block of code a set number of times. When we have a list of things to loop through, we can construct a definite loop using a for statement.

The syntax for the for loop is :

```
for iteration_variable in sequence :  
    statement(s)
```

The *for loop* starts with the *for keyword* and ends with a colon. The first item in the sequence gets assigned to the iteration variable *iteration_variable*. Here, *iteration_variable* can be any valid variable name. Then the statement block is executed. This process of assigning items from the sequence to the *iteration_variable* and then executing the statement continues until all the items in the sequence are completed.

In Python, understanding the for loop is incomplete without knowledge of the *range()* function, a built-in function. It is very useful in demonstrating the *for loop*. The *range()* function generates a sequence of numbers that you can iterate through with a *for loop*. the syntax for *range()* function is,

```
range([start ,] stop [, step])
```

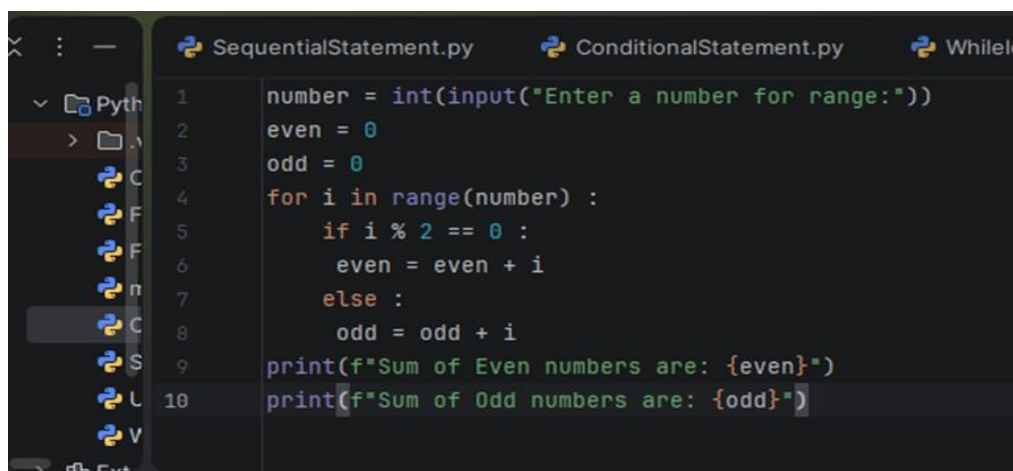
Both start and step arguments are optional and are represented with the help of square brackets, and the range argument value should always be an integer.

start: This value indicates the beginning of the sequence. If the start argument is not specified, the sequence starts at 0 by default.

stop: It generates numbers up to this value but not including the number itself.

step: It indicates the difference between every two consecutive numbers in the sequence. The step value can be negative or positive, but not zero.

Example: Program to find the sum of all odd and even numbers up to a number specified by the user.



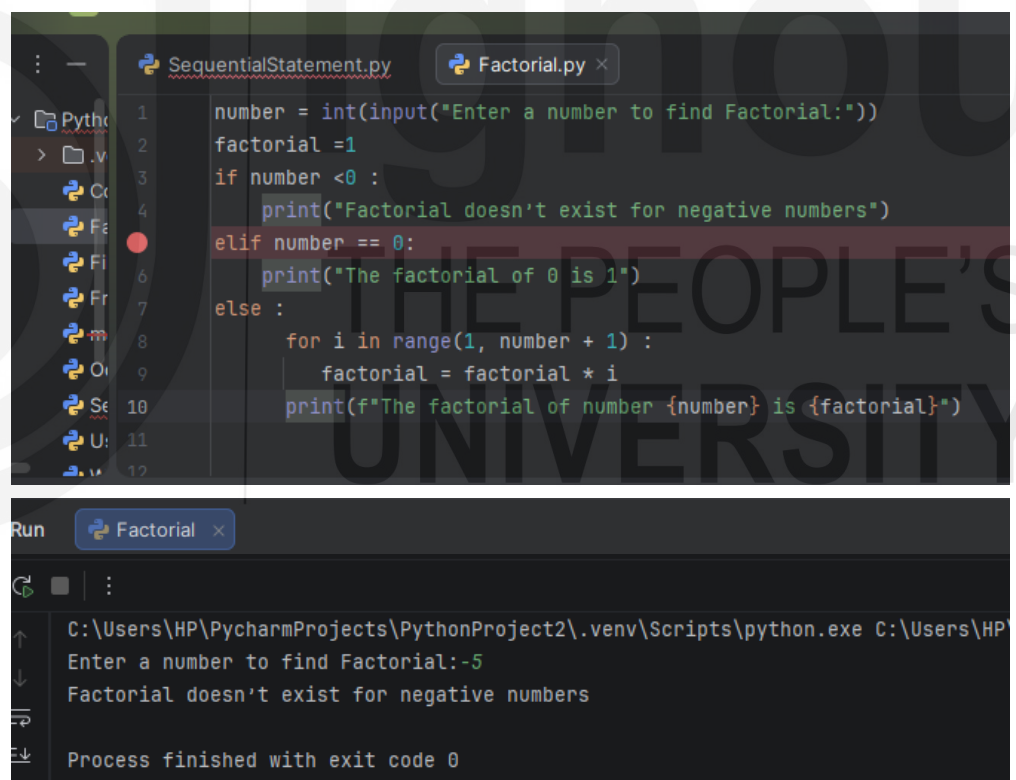
```
1 number = int(input("Enter a number for range:"))  
2 even = 0  
3 odd = 0  
4 for i in range(number) :  
5     if i % 2 == 0 :  
6         even = even + i  
7     else :  
8         odd = odd + i  
9 print(f"Sum of Even numbers are: {even}")  
10 print(f"Sum of Odd numbers are: {odd}")
```



```
Run OddEvenSum x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe
Enter a number for range:25
Sum of Even numbers are: 156
Sum of Odd numbers are: 144
Process finished with exit code 0
```

In the above execution of the program, the value of number is given as 25. A range of numbers is generated using the range() function. Because the range() function only specifies the stop value, it generates numbers from 0 to 24. Then the generated numbers are separated into odd and even using the modulus operator in the for loop. All even numbers are added and assigned to a variable named even, and all odd numbers are added and assigned to a variable named odd; finally, the result is printed. The for loop will be executed/iterated the number of times entered by the user, which is 25 in this case.

Example: Program to find the factorial of a number.



```
SequentialStatement.py Factorial.py x
1 number = int(input("Enter a number to find Factorial:"))
2 factorial = 1
3 if number < 0 :
4     print("Factorial doesn't exist for negative numbers")
5 elif number == 0:
6     print("The factorial of 0 is 1")
7 else :
8     for i in range(1, number + 1) :
9         factorial = factorial * i
10    print(f"The factorial of number {number} is {factorial}")
11
12

Run Factorial x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP
Enter a number to find Factorial:-5
Factorial doesn't exist for negative numbers
Process finished with exit code 0
```

In the above screenshot, you can see that for -5 input, the program to find the factorial is executed, and it displays: Factorial doesn't exist for negative numbers

When you execute this program for, let us say, input 6, you will get the following output/result:

```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe
Enter a number to find Factorial:6
The factorial of number 6 is 720

Process finished with exit code 0
```

If you look at the code, the first line reads the number from the user. A value of 1 is assigned to the variable factorial. To find the factorial of a number, it has to be checked for a non-negative integer. If the user-entered number is zero, then the factorial is 1. To generate numbers from 1 to the user-entered number, the range() function is used. Every number is multiplied by the factorial variable and is assigned to the factorial variable inside the for loop. The for loop block repeats for each number from 1 up to the user-entered number. Finally, it prints the factorial value.

Nested for Loops

A nested loop is a loop inside another loop. In the nested loop, the "inner loop" is executed one time for each iteration of the "outer loop":

Example: Program to print each adjective for every fruit :

```
adj=["red", "big", "tasty"]
fruits=["apple", "banana", "cherry"]
```

```
for x in adj:
    for y in fruits:
        print(x, y)
```

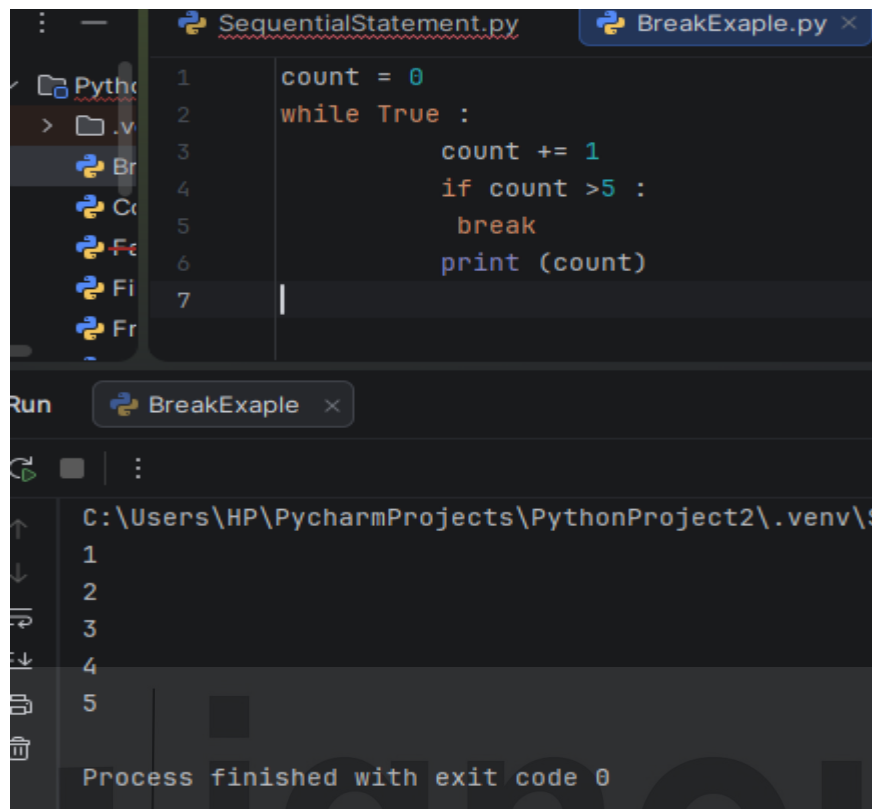
Output:

```
C:\Users\JAY PRAKASH\Desktop\Python\Example>python ex1.py
red apple
red banana
red cherry
big apple
big banana
big cherry
tasty apple
tasty banana
tasty cherry
```

3.3.3.3 The continue and break Statements

The continue and break statements provide greater control over the execution of code in a loop. Whenever the break statement is encountered, the execution control immediately jumps to the first instruction following the loop. To pass control to the next iteration without exiting the loop, the *continue statement* is used. You can use both continue and break statements in while and for loops.

Example: Program that prints integers from zero to 5 using the break statement.



```
1 count = 0
2 while True :
3     count += 1
4     if count > 5 :
5         break
6     print (count)
7
```

Run BreakExaple x

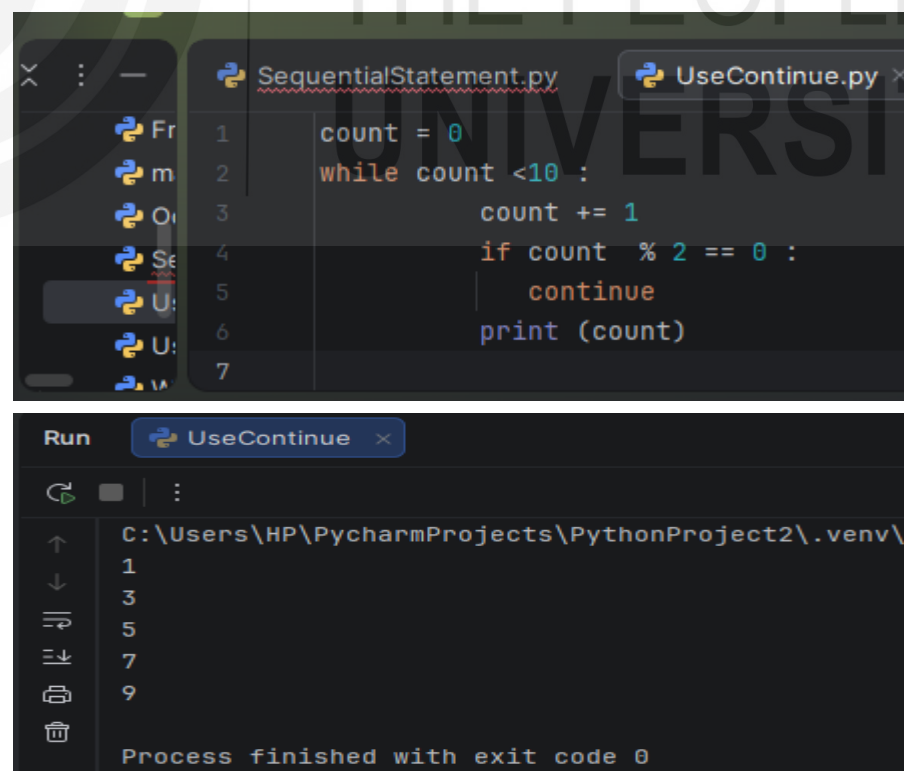
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe

1
2
3
4
5

Process finished with exit code 0

In the above code, in the while loop, as soon as the count reaches 6, the break statement prevents the print statement from executing, and control exits the while loop.

Example: Program that processes only odd integers from 0 to 10 using continue statement.



```
1 count = 0
2 while count < 10 :
3     count += 1
4     if count % 2 == 0 :
5         continue
6     print (count)
7
```

Run UseContinue x

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe

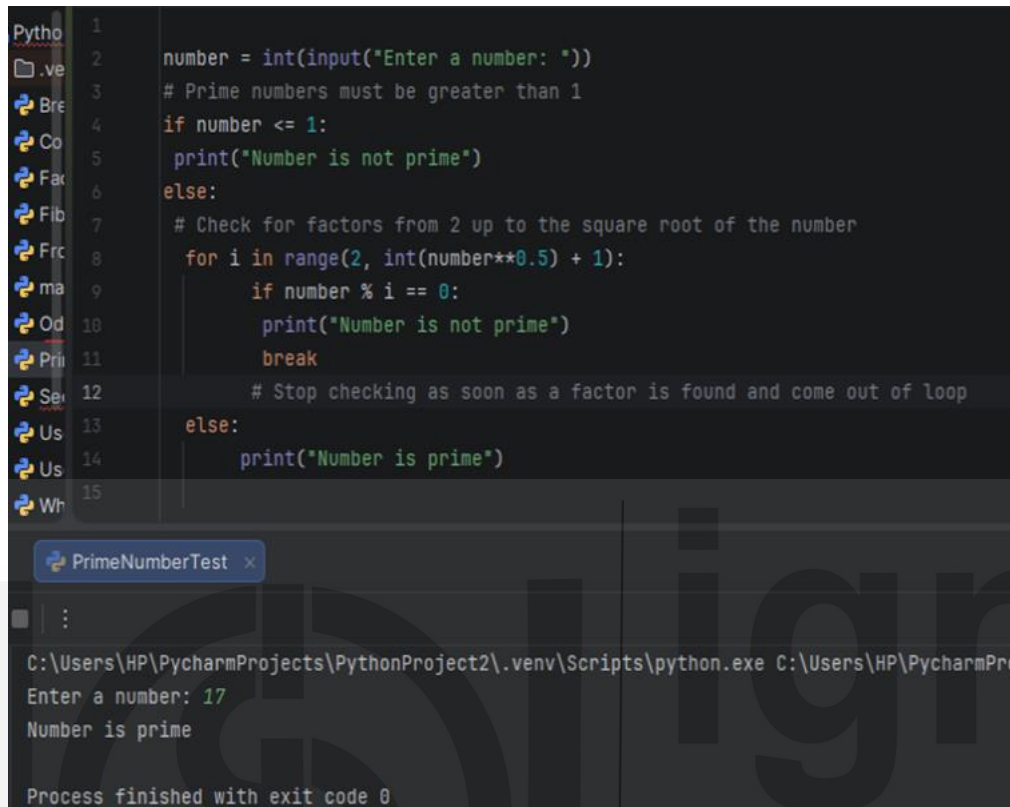
1
3
5
7
9

Process finished with exit code 0

The continue statement in the while loop will not let the print statement to execute if the count value is even ,and control will go for next iteration of the

loop. Here, the loop will go through all the iterations, and the *continue statement* will affect only the statements in the loop are to be executed or not occurring after the *continue statement*.

Example: Program to check whether a number is prime or not.



```

1  number = int(input("Enter a number: "))
2  # Prime numbers must be greater than 1
3  if number <= 1:
4      print("Number is not prime")
5  else:
6      # Check for factors from 2 up to the square root of the number
7      for i in range(2, int(number**0.5) + 1):
8          if number % i == 0:
9              print("Number is not prime")
10             break
11         # Stop checking as soon as a factor is found and come out of loop
12     else:
13         print("Number is prime")
14
15

```

PrimeNumberTest x

```

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmPro
Enter a number: 17
Number is prime
Process finished with exit code 0

```

Check your progress-2

Q1). In a Python program, a control structure:

- Defines program-specific data structures
- Directs the order of execution of the statements in the program
- Dictates what happens before the program starts and after it terminates
- None of the above

2). Which of the following is False regarding loops in Python?

- Loops are used to perform certain tasks repeatedly
- A while loop is used when multiple statements are to be executed repeatedly until the given condition becomes False
- A while loop is used when multiple statements are to be executed repeatedly until the given condition becomes True.
- A for loop can be used to iterate through the elements of lists.

3). We can write if/else into one line in Python.

- True
- False

Q4). Given the nested if-else below, what will be the value of x when the code is executed successfully?

```

x = 0
a = 5
b = 5

```

```
if a > 0:
    if b < 0:
        x = x + 5
elif a > 5:
    x = x + 4
else:
    x = x + 2
print(x)
```

- a) 0
- b) 4
- c) 2
- d) 3

Q5). if -3 evaluates to:

- a) True
- b) False

Q6). What is the output of the following nested loop

```
numbers = [10, 20]
items = ["Chair", "Table"]
for x in numbers:
    for y in items:
        print(x, y)
```

- a) 10 Chair
10 Table
20 Chair
20 Table
- b) 10 Chair
20 Chair
10 Table
20 Table

Q7). What is the value of X?:

```
X = 0
```

```
while (X < 100):
```

```
    X+=2
```

```
print(x)
```

- a) 101
- b) 99
- c) None of the above, this is an infinite loop
- d) 100

Q8). What is the value of the var after the for loop completes its execution?

```
var = 10
```

```
for i in range(10):
```

```
    for j in range(2, 10, 1):
```

```
        if var % 2 == 0:
```

```
            continue
```

```
            var += 1
```

```
        var+=1
```

```
else:
```

```
    var+=1
```

print(var)

- a) 20
- b) 21
- c) 10
- d) 30

Q9). Find the output of the following program:

```
a = {i :i * i for i in range(6)}
```

```
print (a)
```

- a) Dictionary comprehension doesn't exist
- b) {0:0, 1:1, 2:4, 3:9, 4:16, 5:25, 6:36}
- c) {0:0, 1:1, 4:4, 9:9, 16:16, 25:25}
- d) {0:0, 1:1, 2:4, 3:9, 4:16, 5:25}

10) Write the benefits of using iterations or loops in Python.

11) What are range functions? Why do we need range functions in a for loop?

12) Write Python code using a for loop that outputs all the multiples of 5 between 108 and 205.

3.4 SUMMARY

This unit explains how sequential and control flow statements are used in Python. All the basic structures of Python can be combined with control flow statements, which ultimately form a Python program. This unit explained how to use if, if..else, if..elif, and looping statements like for and while with several programming examples. However, this is not all; there is still a long way to go in the Python programming journey. Python has many more unique features which make it a programmer's language.

3.5 SOLUTION/ANSWER TO CHECK YOUR PROGRESS

Answer to Check your progress-1

1.



```
1 # Program to check whether the given number is positive/negative and even/odd
2 num = (int)(input("Enter the number: "))
3 # Check whether number is positive, negative, or zero
4 if num > 0:
5     sign = "Positive"
6     if num%2 == 0:
7         print ( "The number entered is Even and", sign)
8     else:
9         print("The number entered is Odd and", sign)
10 elif num < 0:
11     sign = "Negative"
12     if num%2 == 0:
13         print ( "The number entered is Even and", sign)
14     else:
15         print("The number entered is Odd and", sign)
16 else:
17     sign = "Zero (neither positive nor negative)"
18     print(sign)
```

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\Scripts\python.exe

Enter the number: 45

The number entered is Odd and Positive

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\Scripts\python.exe

Enter the number: -67

The number entered is Odd and Negative

Process finished with exit code 0

2.

Program for bonus calculation. Assuming different rates of DA, HRA and bonus for different grades

Taking input from the user

emp_id = input("Enter Employee ID: ")

name = input("Enter Employee Name: ")

basic_pay = float(input("Enter Basic Pay: "))

grade = input("Enter Grade (A, B, or C): ").upper()

Calculating DA and HRA using if-elif-else statements

if grade == 'A':

da = 0.50 * basic_pay

hra = 0.30 * basic_pay

```

        bonus_percentage = 0.20
elif grade == 'B':
    hra = 0.20 * basic_pay
    da = 0.40 * basic_pay
    bonus_percentage = 0.15
elif grade == 'C':
    da = 0.30 * basic_pay
    hra = 0.10 * basic_pay
    bonus_percentage = 0.10
else:
    print("Invalid Grade entered!By default Grade is C.")
    grade = 'C'
    da = 0.30 * basic_pay
    hra = 0.10 * basic_pay
    bonus_percentage = 0.10
# Calculating Gross Salary and Bonus
gross_salary = basic_pay + da + hra
bonus = gross_salary * bonus_percentage
# Displaying the results
print("="*40)
print(f"PAYSLIP FOR EMPLOYEE: {name} (ID: {emp_id})")
print("=" * 40)
print(f"Grade           : {grade}")
print(f"Basic Pay        : ₹{basic_pay:,.1f}")
print(f"DA               : ₹{da:,.1f}")
print(f"HRA              : ₹{hra:,.1f}")
print(f"Gross Salary     : ₹{gross_salary:,.1f}")
print(f"Bonus           : ₹{bonus:,.1f}")
print("=" * 40)

```

Output of the Program:

```

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects
Enter Employee ID: 125
Enter Employee Name: Mohan
Enter Basic Pay: 50000
Enter Grade (A, B, or C): A
=====
PAYSLIP FOR EMPLOYEE: Mohan (ID: 125)
=====
Grade           : A
Basic Pay        : ₹50,000.0
DA               : ₹25,000.0
HRA              : ₹15,000.0
Gross Salary     : ₹90,000.0
Bonus           : ₹13,500.0
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\Py
=====
Enter Employee ID: 125
Enter Employee Name: Mohan
Enter Basic Pay: 50000
Enter Grade (A, B, or C): A
Process fin:

```

```
C:\Users\DELL\PycharmProjects\PythonProject\.venv\Scripts\python.exe
Enter Employee ID: 1
Enter Employee Name: mp
Enter Basic Pay: 50000
Enter Grade (A, B, or C): A

-----
PAYSIP FOR EMPLOYEE: mp (ID: 1)
-----

Grade           : A
Basic Pay        : ₹50,000.00
DA               : ₹25,000.00
HRA              : ₹15,000.00
Gross Salary    : ₹90,000.00
Bonus           : ₹18,000.00
-----

Process finished with exit code 0
```

Answer to Check your progress-2

1. b
2. b
3. c
4. a
5. a
6. a
7. d
8. b
9. d

10.

Loops make Python programs simpler, more efficient, and more powerful. Iteration and loops (for and while) are fundamental structures in Python that allow you to execute a block of code repeatedly. In Python, iteration allows you to traverse and manipulate data structures of variable lengths (like lists, dictionaries, tuples, and sets) without needing to know their exact size in advance. They help handle variable-sized datasets dynamically. They also help create clear code structure.

11.

The range() function is essential in Python for creating number sequences and controlling loops efficiently, making programs cleaner, faster, and easier to understand.

12.

```
for num in range(108,205,1):
    if(num%5==0):
        print(num)
```

3.6 REFERENCES/FURTHER READINGS

1. Yashavant Kanetkar and Aditya Kanetkar, “Let Us Python”, 4th Edition, BPB Publications, New Delhi , 2022
2. Paul Deitel and Harvey Deitel , “Python For Programmers” , 1st Edition, Pearson India Education , Chennai, 2023
3. Sheetal Taneja and Naveen Kumar, “Python Programming a Modular Approach” , 1st Edition, Pearson India Education , Chennai , 2022
4. S. Shridhar, J. Indumathi and V.M. Hariharan , “Python Programming” , 1st Edition, Pearson India Education , Chennai, 2024
5. S. Suma Christal Mary , R. Josphine Leela, N. Senthamilarasi and Y. Chrisbin Jeeva , “Problem Solving and Python Programming”, 1st Edition, Ane Books , New Delhi, 2018

